



UNIVERSITETI[®]
METROPOLITAN
TIRANA

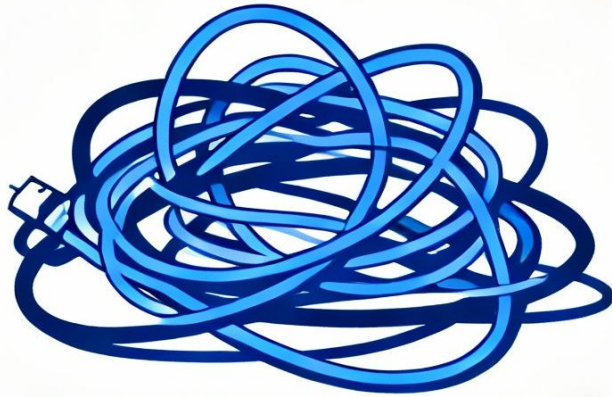
Course: Object Oriented Programming

Thinking in Things

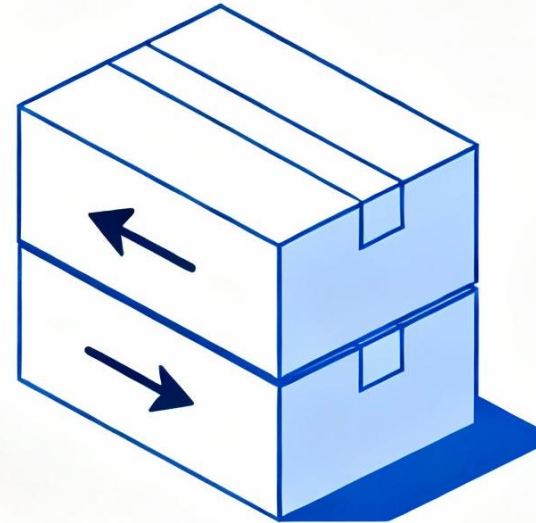
Seeing a problem as objects

Evis Plaku

Same problem, cut a different way



Procedural: steps everywhere

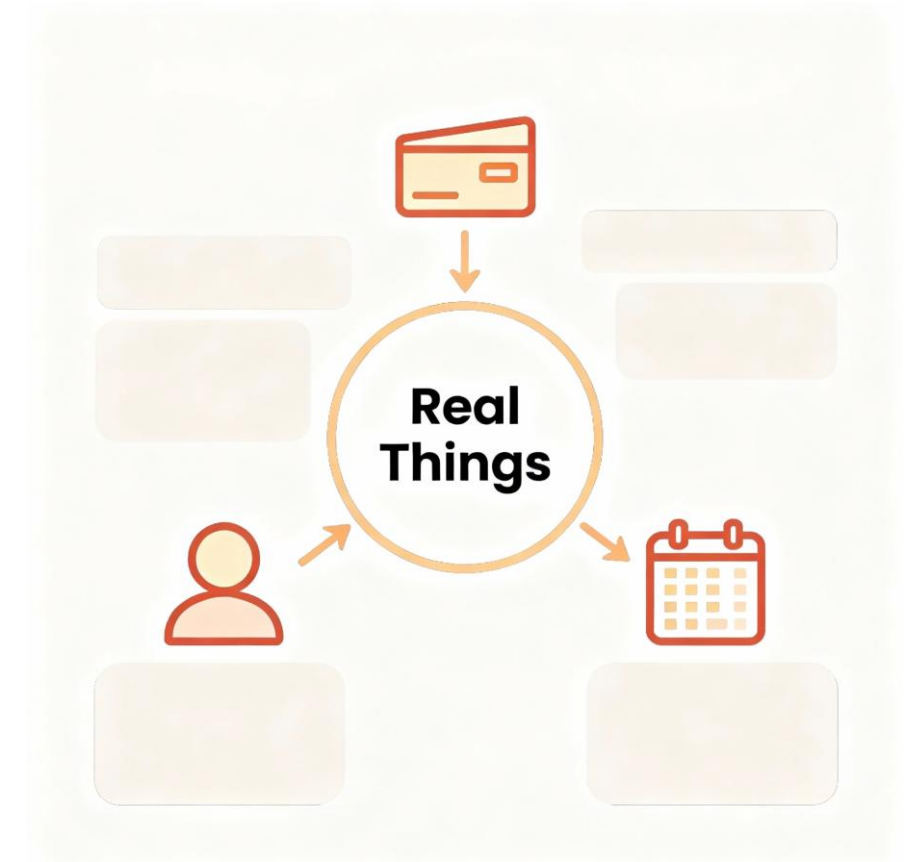


OOP: parts talk



Learn to design software that survives change

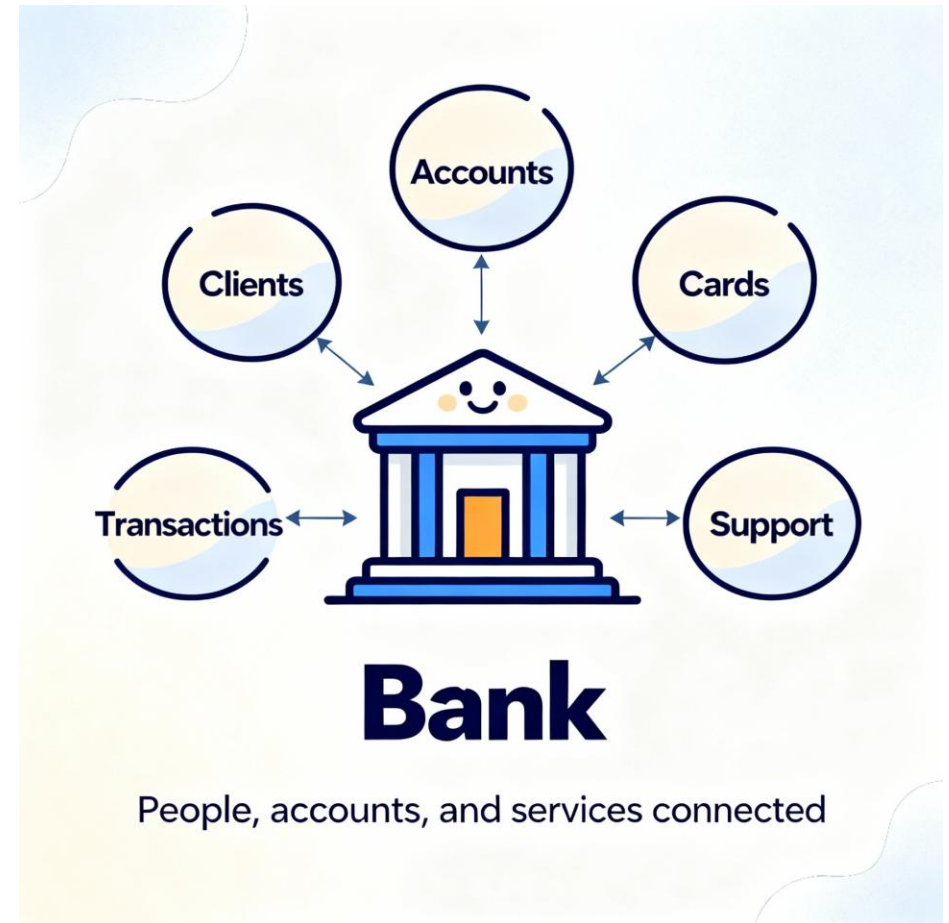
- You write code that runs. This is about code that keeps working
- Four modules, one application, built in three sprints
- No Java today. Today is a way of seeing





A bank management domain to anchor ideas and connect weekly topics

- We will model entities, rules, and actions
- We will refine flows step by step as concepts deepen

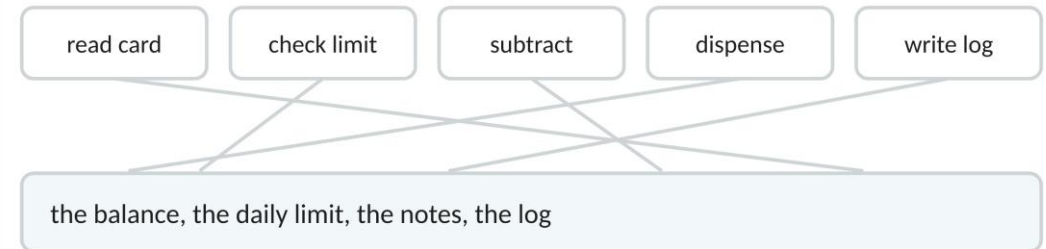




A cash machine, described two ways

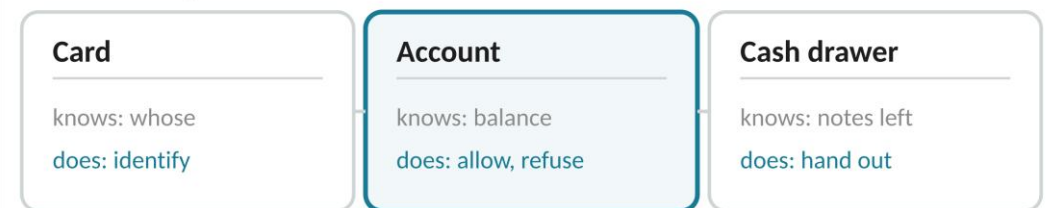
- As steps: read the card, check the limit, subtract, dispense
- As things: a card, an account, a drawer, each deciding its part
- Nothing added, nothing removed. Only the boundaries moved

Cut into steps



Every step needs the data, so the data belongs to nobody in particular.

Cut into things



Each thing keeps what it knows and decides what it does.



It **knows** something, and it **does** something

- What it knows is its state. What it does is its behaviour
- One thing holds both, and the rest of the course is detail
- A thing that can say no is an object

State



Behavior



Request → Response



Who should **know** this, and who should **decide**?

- Take any rule: a withdrawal cannot exceed the balance
- The account owns the balance, so the account decides
- Where does this code go stops being a matter of taste

the rule

A withdrawal cannot exceed the balance

Who knows the balance?

Cashier

counts notes, not money



Log

records what happened



Account

holds the balance itself



so the account is the one that decides



Three questions, **not underlined nouns**

- Does it have data of its own that must stay correct?
- Are there rules about that data, and who enforces them?
- Can you ask it to do something, and can it say no?

Knows

Does

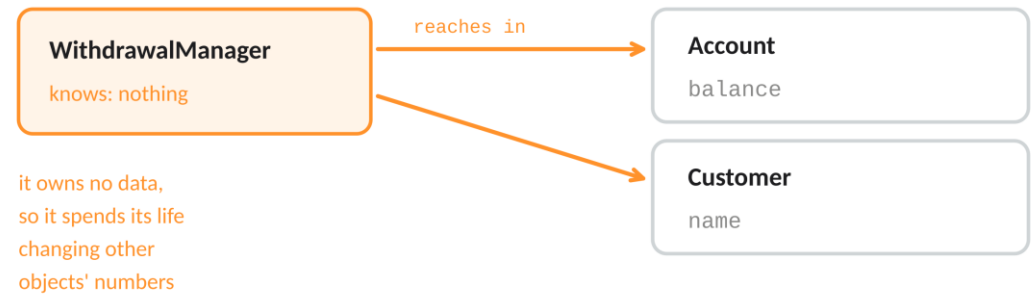
Refuses



Verbs are usually **methods**, not classes

- WithdrawalManager, TransactionHandler, BalanceProcessor
- Ask what it knows. Usually nothing, so it reaches into others
- A step wearing a noun's name, and you are back where you started

A step wearing a noun's name



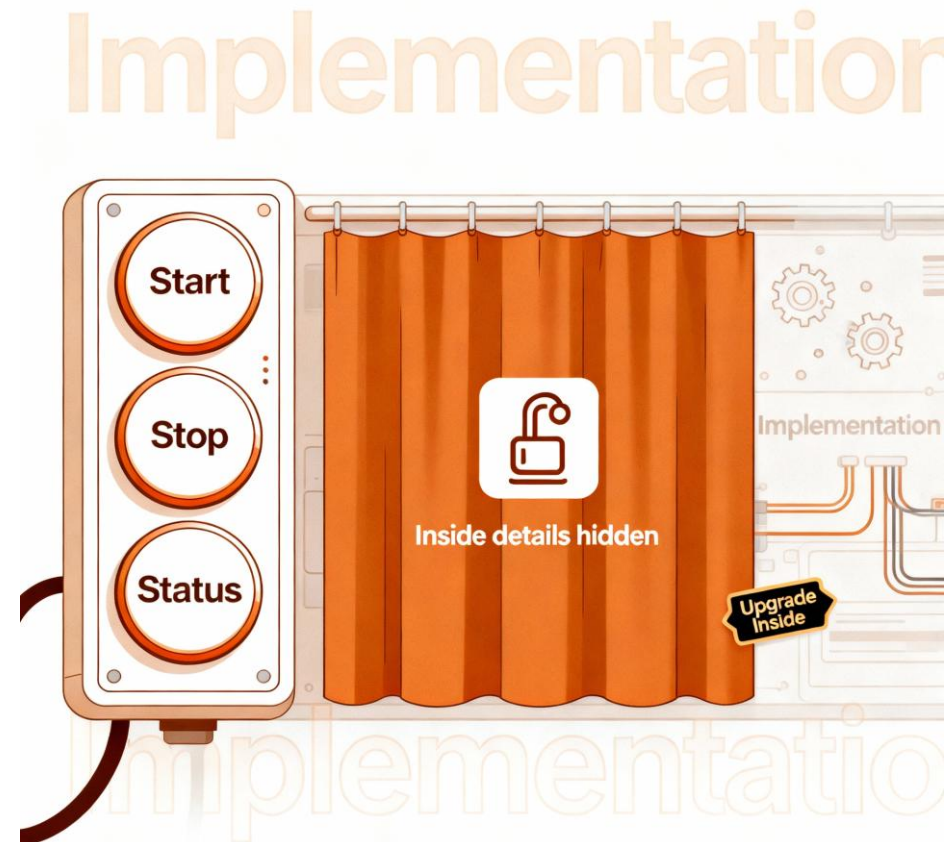
The behaviour belongs to whoever owns the data





Ask, do not take

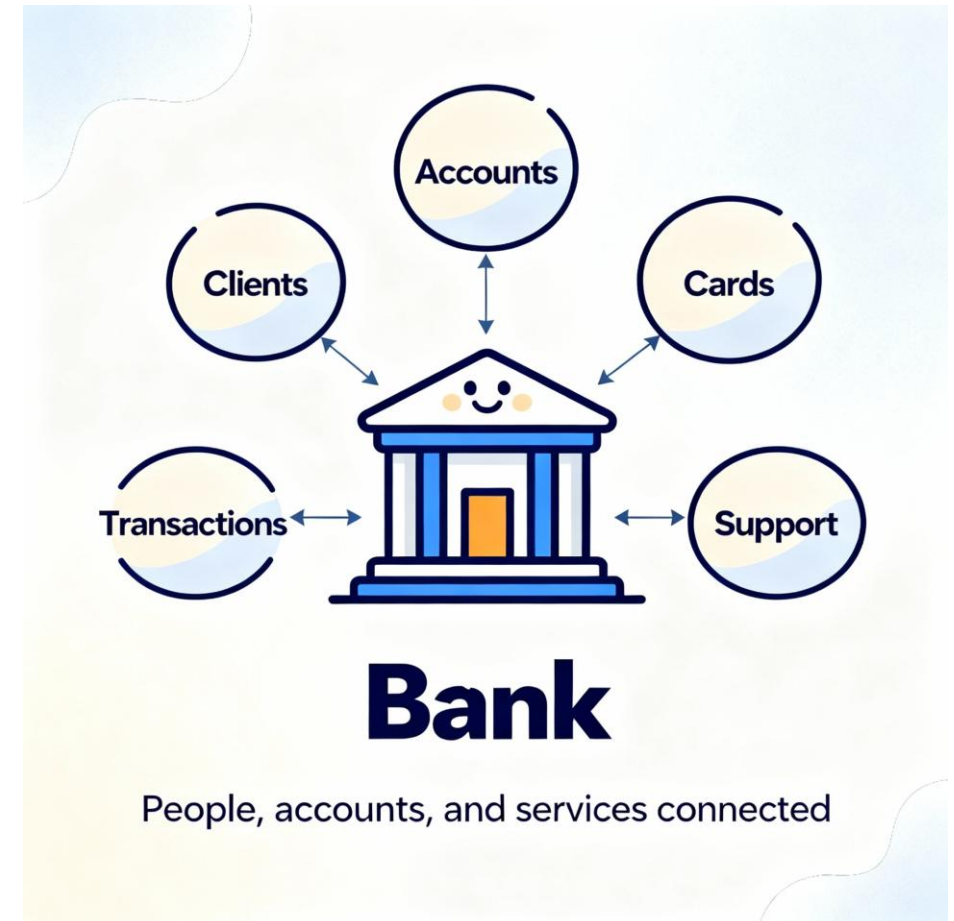
- You do not reach into a wallet. You ask, and the owner decides
- `account.withdraw(50)`, never `account.balance = balance - 50`
- The account can change how it works and nobody outside notices





A **bank**, because everyone already knows the rules

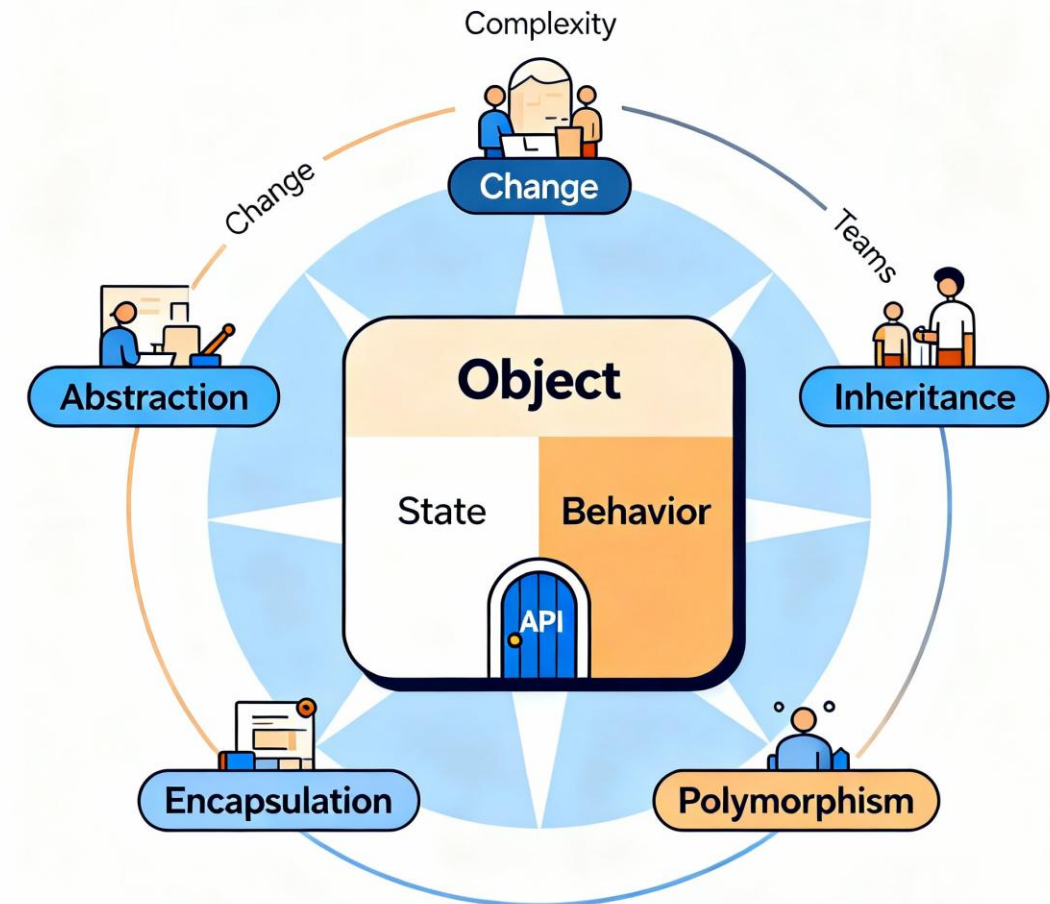
- A balance cannot go negative. An account number never changes
- Money that leaves one place has to arrive somewhere
- Strict rules are exactly what objects are for

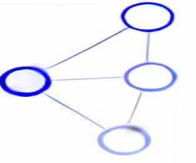




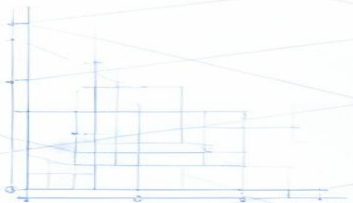
An object **knows something** and can be **asked to do something**

- Designing is deciding who knows what, and so who decides what
- Objects model the problem, not the world
- Next: what goes wrong when a rule lives away from its data





Why objects





A program *you* already wrote

- Several lines of C, it works, and nothing is wrong with it
- Names in one array, grades in another
- It stays fine for about a week

```
char  names[100][50];
float grades[100];
int   count = 0;

void add_student(const char *name,
                 float grade) {
    strcpy(names[count], name);
    grades[count] = grade;
    count++;
}

float average(void) {
    float total = 0;
    for (int i = 0; i < count; i++)
        total += grades[i];
    return total / count;
}
```



Where is the **rule** in this program?

- Grades must be 0 to 10. The check goes into `add_student`
- Then file import. Second copy. Then corrections. Third
- Then one line at 2am skips all three, and it compiles

```
// three copies of one rule
if (g < 0 || g > 10) return;

// ...and then, somewhere else
grades[3] = 11.0;

// no check was consulted,
// because checks live in
// functions and this line
// did not call one
```



The failure that **never announces itself**

- Drop a student. Shift names down, shift grades down
- Next month somebody adds emails[100] and forgets it
- No crash, no error. Just the wrong email from then on

```
// remove student 3
for (int i = 3; i < count-1; i++) {
    strcpy(names[i], names[i+1]);
    grades[i] = grades[i+1];
    // emails[i] = emails[i+1];
    // <- forgotten
}
count--;
```



You split the program by **what it does**

- Every function needs the data, so all of them can reach it
- Reachable by everything means changeable by everything
- When everyone shares responsibility, nobody has it

Before removing student 3

names	Ada	Alan	Grace	Linus
grades	9.1	7.4	8.8	6.0
emails	ada@	alan@	grace@	linus@

Index 2 is Grace in all three. Only because everyone remembered.

Two arrays shifted, one forgotten

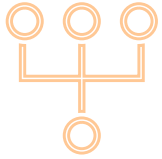
names	Ada	Alan	Linus
grades	9.1	7.4	6.0
emails	ada@	alan@	grace@



You already half solved this in C

- A struct holds the data together. One student, one thing
- Delete students[3] and name and grade go together
- But it defends nothing. The next line still compiles

```
struct Student {  
    char  name[50];  
    float grade;  
};  
  
struct Student students[100];  
  
// still compiles, still runs  
students[3].grade = 11.0;
```



C gives you the pieces. Java enforces how they fit

- Behaviour is attached to the thing, so a request can arrive
- The compiler decides who may touch the data
- private stops being a convention and becomes a guarantee

C already gives you

```
struct Student { ... };
```

the data is grouped, and cannot drift apart



```
students[3].grade = 11.0;
```

still compiles. the struct holds, it does not defend



What Java adds

The behaviour is attached to the thing

a request has somewhere to arrive

The compiler decides who may touch it

private stops being a convention

One request, different code

week seven



One place **subtracts** from a balance

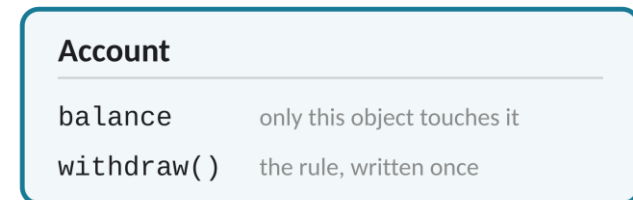
- The rule now lives beside the data it protects
- One place changes when the overdraft policy arrives
- No index to keep in step, because an account is one thing

Data here, rules over there



One rule, written three times. Nothing stops a fourth place forgetting it.

Data and its rules together



The rule cannot be skipped, because it is part of the thing it protects.



Ten lines, and two ideas

- balance is private, so `account.balance -= 5000` will not compile
- withdraw can refuse, and it reports rather than prints

```
public class Account {  
    private double balance;  
  
    public Account(double opening) {  
        this.balance = opening;  
    }  
  
    public boolean withdraw(double a) {  
        if (a <= 0 || a > balance) {  
            return false;  
        }  
        balance -= a;  
        return true;  
    }  
}
```



Most nouns are **not objects**

- Own data, rules about that data, and something you can ask it to do

Classes

Account owns a balance, has rules, can be asked	✓
Customer owns a name and a set of accounts	✓
Transaction a record of what happened	✓

Three questions decide it: own data, rules about it, and something you can ask it to do.

Not classes

Balance	a number that belongs to an account	✗
Interest rate	a value, until the rules get complicated	✗
Total holdings	calculated from accounts, not a thing	✗
Withdrawing	something an account does	✗

The last one is where people go wrong. **Withdrawing** feels important, so it becomes a **WithdrawalManager**.

Ask. Do not take.

That's manners in life, and messages in OOP.

